

The problem: detect meaningful shelf events

INPUT CSV readings time_seconds,weights 0.000,72.05 0.005,72.14 Each row records when the scale measured and what it measured.	GOAL Find real changes Separate a sustained item removal or return from brief movement, handling, or sensor noise.	OUTPUT Signed events (-69.87, 1.326) (+69.29, 2.466) Negative means removed. Positive means added or returned.
---	---	---

Central question: has the shelf settled at a new weight level, or was this only a temporary spike?

The project has one clear processing pipeline

01 CSV input read_measurements() Reads rows and creates "Measurement" objects. →	02 Validation Rejects missing, invalid, or badly ordered data. →	03 Detection detect_weight_change_details() Runs the two-state algorithm. →	04 Results detect_weight_change() Returns rounded event tuples. →	05 CLI / plots main() prints results; optional Matplotlib plots show events.
TEST LANE test_weight_event_detector.py independently checks all three recordings: exactly two chronological events, removal first, return second, and expected magnitudes.				

Complete algorithm in one compact flow

1. Read and validate the CSV.
2. Calculate sensor noise once for the file.
3. Calculate adaptive thresholds.
4. Calculate the initial baseline from the first 0.15 seconds.
5. Start in Stable mode.
6. Compare each new reading with the current baseline.
7. If the difference exceeds the activity threshold, start Activity mode.
8. Save the first activity timestamp.
9. Collect readings in a sliding 0.15-second window.
10. Wait until the window covers at least 95% of the required duration.
11. Check whether the window's weight range is small enough.
12. Calculate the median of the stable window.
13. Compare the new median with the previous baseline.
14. Record an event only if the change exceeds the change threshold.
15. Update the baseline regardless of whether an event was recorded.
16. Return to Stable mode and continue.

The main algorithm is contained primarily in **lines 245–334**, while the supporting functions are located in **lines 115–242**.

Algorithm steps with exact line numbers

The line numbers below refer to the current `weight_event_detector.py` file in your Docker project.

Step 0 — Define the data and configuration

Lines 28–34: Measurement stores one timestamp and one weight value.

```
@dataclass(frozen=True)
class Measurement:
    time_seconds: float
    weight_grams: float
```

Lines 36–50: DetectionParameters defines the algorithm settings:

```
initial_baseline_duration_seconds: float = 0.15
confirmation_duration_seconds: float = 0.15
required_confirmation_fraction: float = 0.95
minimum_activity_threshold_grams: float = 5.0
minimum_change_threshold_grams: float = 5.0
minimum_stability_tolerance_grams: float = 2.5
```

In simple words, the algorithm looks at approximately 0.15 seconds of data for the initial baseline and for confirming a new stable level. It requires at least 95% of the confirmation duration.

Step 1 — Read and validate the CSV file

Lines 61–112: read_measurements()

First, the code checks that the file exists:

```
input_path = Path(csv_path)

if not input_path.is_file():
    raise FileNotFoundError(
        f"Input file does not exist: {input_path}"
    )
```

Lines 76–83 check that the CSV contains the required columns:

```
reader = csv.DictReader(input_file)
required_columns = {"time_seconds", "weights"}

if reader.fieldnames is None or not
required_columns.issubset(reader.fieldnames):
    raise ValueError(
        "The CSV file must contain the columns 'time_seconds' and 'weights'."
    )
```

Lines 85–97 convert the values to floating-point numbers and reject invalid values:

```
try:
    time_seconds = float(row["time_seconds"])
    weight_grams = float(row["weights"])
except (TypeError, ValueError) as error:
    raise ValueError(
        f"Row {row_number} contains a non-numeric time or weight value."
    ) from error

if not math.isfinite(time_seconds) or not math.isfinite(weight_grams):
    raise ValueError(
        f"Row {row_number} contains a non-finite time or weight value."
    )
```

Lines 99–112 ensure that time increases and that there are enough measurements:

```
if measurements and time_seconds <= measurements[-1].time_seconds:
    raise ValueError(
        "The time_seconds column must be strictly increasing."
    )

if len(measurements) < 2:
    raise ValueError(
        "At least two measurements are required for event detection."
    )
```

Purpose: Bad data is rejected before it can produce a misleading event.

Step 2 — Calculate the median when needed

Lines 115–126: `median_weight()`

```
def median_weight(measurements: Iterable[Measurement]) -> float:
    weights: list[float] = []

    for measurement in measurements:
        weights.append(measurement.w      eight_grams)

    if not weights:
        raise ValueError(
            "Cannot calculate a median from an empty measurement set."
        )

    return statistics.median(weights)
```

The function extracts the weights and sends them to `statistics.median()`.

The median is used in two important places:

1. To calculate the initial shelf baseline.
 2. To calculate the new shelf weight after the scale settles.
-

Step 3 — Calculate the initial baseline

Lines 227–242: `choose_initial_baseline()`

```
def choose_initial_baseline(
    measurements: Sequence[Measurement],
    parameters: DetectionParameters,
) -> float:
    starting_time = measurements[0].time_seconds
    initial_window: list[Measurement] = []
    final_initial_window_time = (
        starting_time + parameters.initial_baseline_duration_seconds
    )

    for measurement in measurements:
        if measurement.time_seconds <= final_initial_window_time:
            initial_window.append(measurement)
```

```
return median_weight(initial_window)
```

The calculation works like this:

1. Take the time of the first measurement.
2. Add 0.15 seconds.
3. Collect all measurements inside that period.
4. Calculate their median.
5. Store that median as the first `resting_weight_grams` value.

The initial baseline is created at **line 274**:

```
resting_weight_grams = choose_initial_baseline(
    measurements, parameters
)
```

This is the weight against which the first possible event is compared.

Step 4 — Estimate sensor noise

Lines 154–172: `estimate_sensor_noise_grams()`

```
def estimate_sensor_noise_grams(
    measurements: Sequence[Measurement]
) -> float:
    absolute_differences: list[float] = []

    for index in range(1, len(measurements)):
        previous_measurement = measurements[index - 1]
        current_measurement = measurements[index]
        difference_grams = abs(
            current_measurement.weight_grams
            - previous_measurement.weight_grams
        )
        absolute_differences.append(difference_grams)

    return statistics.median(absolute_differences)
```

The code compares each measurement with the measurement immediately before it. It calculates the absolute differences and then takes their median.

This value is calculated once for the file at **line 267**:

```
sensor_noise_grams = estimate_sensor_noise_grams(measurements)
```

In the current version, sensor noise is not recalculated after every event.

Step 5 — Calculate adaptive thresholds

Lines 175–197: `calculate_thresholds()`

```

activity_threshold_grams = max(
    parameters.minimum_activity_threshold_grams,
    12.0 * sensor_noise_grams,
)

change_threshold_grams = max(
    parameters.minimum_change_threshold_grams,
    12.0 * sensor_noise_grams,
)

stability_tolerance_grams = max(
    parameters.minimum_stability_tolerance_grams,
    6.0 * sensor_noise_grams,
)

```

The values are created at **lines 268–272**:

```

(
    activity_threshold_grams,
    change_threshold_grams,
    stability_tolerance_grams,
) = calculate_thresholds(
    sensor_noise_grams, parameters
)

```

The rules are:

Purpose	Calculation	Threshold
Starts activity detection.	$\max(5.0, 12 \times \text{noise})$	Activity threshold
Decides whether to record an event.	$\max(5.0, 12 \times \text{noise})$	Change threshold
Decides whether a window is settled.	$\max(2.5, 6 \times \text{noise})$	Stability tolerance

Step 6 — Start in the stable state

Lines 274–278 initialize the detector:

```

resting_weight_grams = choose_initial_baseline(measurements, parameters)
events: list[DetectionResult] = []
activity_in_progress = False
event_start_time_seconds: float | None = None
activity_measurements: list[Measurement] = []

```

The important variable is:

```
activity_in_progress = False
```

`False` means the detector currently believes the shelf is stable and is watching for a meaningful departure.

Step 7 — Detect the beginning of possible activity

The main loop starts at **line 280**:

```
for measurement in measurements:
```

Lines 281–291 implement the stable state:

```
if not activity_in_progress:
    deviation_from_resting_weight = abs(
        measurement.weight_grams - resting_weight_grams
    )

    if deviation_from_resting_weight >= activity_threshold_grams:
        activity_in_progress = True
        event_start_time_seconds = measurement.time_seconds
        activity_measurements = [measurement]

    continue
```

For every new reading, the code calculates:

```
absolute(current weight - current resting baseline)
```

If the result is at least the activity threshold:

1. Activity begins.
2. The exact time is saved in `event_start_time_seconds`.
3. The detector switches to activity mode.
4. The current measurement starts the activity list.

At this stage, the program has **not yet confirmed an event**. It has only detected that something may have happened.

Step 8 — Collect readings while activity is in progress

Lines 293–298 collect new readings and keep only the recent time window:

```
activity_measurements.append(measurement)
activity_measurements = measurements_in_recent_duration(
    activity_measurements,
    parameters.confirmation_duration_seconds,
)
recent_measurements = activity_measurements
```

The helper function is defined at **lines 129–151**:

```
def measurements_in_recent_duration(
    measurements: Sequence[Measurement], duration_seconds: float
) -> list[Measurement]:
    if not measurements:
        return []

    final_time = measurements[-1].time_seconds
    earliest_time = final_time - duration_seconds
```

```

recent_measurements: list[Measurement] = []

for measurement in measurements:
    if measurement.time_seconds >= earliest_time:
        recent_measurements.append(measurement)

return recent_measurements

```

This is the sliding window. It keeps readings from approximately the last 0.15 seconds relative to the newest reading.

Step 9 — Check whether the window is long enough

Lines 300–310 calculate how much time the current window covers:

```

observed_confirmation_duration_seconds = (
    recent_measurements[-1].time_seconds
    - recent_measurements[0].time_seconds
)

required_confirmation_duration_seconds = (
    parameters.confirmation_duration_seconds
    * parameters.required_confirmation_fraction
)

has_confirmation_duration = (
    observed_confirmation_duration_seconds
    >= required_confirmation_duration_seconds
)

```

With the default settings:

required duration = 0.15 × 0.95 = 0.1425 seconds

Lines 312–313 keep waiting if the window is not long enough:

```

if not has_confirmation_duration:
    continue

```

This prevents the detector from declaring a new stable state based on only one or two readings.

Step 10 — Check whether the new window is stable

Lines 315–316 call the stability function:

```

if not is_stable(
    recent_measurements,
    stability_tolerance_grams,
):
    continue

```

The complete `is_stable()` function is at **lines 200–224**:

```

def is_stable(
    measurements: Sequence[Measurement],
    stability_tolerance_grams: float,

```



```

) -> bool:
    if not measurements:
        return False

    weights: list[float] = []

    for measurement in measurements:
        weights.append(measurement.weight_grams)

    peak_to_peak_variation_grams = max(weights) - min(weights)
    allowed_peak_to_peak_variation_grams = (
        2.0 * stability_tolerance_grams
    )

    return (
        peak_to_peak_variation_grams
        <= allowed_peak_to_peak_variation_grams
    )

```

The calculation is:

highest weight in window – lowest weight in window

The window is stable when:

peak-to-peak variation $\leq 2 \times$ stability tolerance

If the readings are still moving too much, the code executes `continue` and waits for later readings.

Step 11 — Calculate the new stable weight

Once the window is long enough and stable, **line 318** calculates the new median:

```

new_resting_weight_grams = median_weight(
    recent_measurements
)

```

Then **line 319** calculates the difference from the previous baseline:

```

weight_change_grams = (
    new_resting_weight_grams - resting_weight_grams
)

```

For example:

Old baseline:	500.00 g
New stable median:	0.20 g
Weight change:	-499.80 g

The negative value means that weight was removed.

Step 12 — Decide whether to record the event

Lines 321–327 apply the final change threshold:

```
if abs(weight_change_grams) >= change_threshold_grams:
    events.append(
        DetectionResult(
            weight_change_grams=weight_change_grams,
            change_time_seconds=event_start_time_seconds,
        )
    )
```

There are two outcomes:

Large final difference

If the absolute difference is at least the change threshold, the code records one event.

Small final difference

If the new stable weight is similar to the old baseline, no event is recorded. This usually means the disturbance ended near the original weight.

However, the detector still updates its baseline in the next step.

Step 13 — Update the baseline and return to stable mode

Lines 329–332 reset the detector:

```
resting_weight_grams = new_resting_weight_grams
activity_in_progress = False
event_start_time_seconds = None
activity_measurements = []
```

This happens even when no event was recorded.

Therefore:

- A new stable weight becomes the reference for the next event.
 - The detector returns to stable mode.
 - The activity list is emptied.
 - The next measurement is compared against the updated baseline.
-